COMPUTER SCIENCE + COMPUTER ENGINEERING

# Module 05

A complete first game

Four expanded lessons with worked examples, practical assignments and five-question quizzes. Use the main workbook for this module’s project and additional exercises.

**4 lessons · 4 demonstrations · 20 lesson questions**

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

# Contents

Use the linked contents or PDF bookmarks to navigate.

**Module 05: A complete first game** ..... 3
Lesson 05.1: The update loop and command model ..... 3
Lesson 05.2: Collision rules and complete behavior ..... 6
Lesson 05.3: Deterministic replay and simple measurement ..... 9
Lesson 05.4: Finishing, usability and the first portfolio artifact ..... 12

# Module 05: A complete first game

**Prerequisites:** Modules 01-04.

**Integration project:** Training Yard: the Gate A first-game release. The full brief and review are in the Lab Workbook.

# Lesson 05.1: The update loop and command model

**Learning outcome:** Separate simulation ticks from input collection and presentation.

## One command, one simulation decision

A turn-based game loop can be described as read a command, decide its effect, produce the next state and display that state. The display may also be refreshed independently, but it must not apply another movement. We choose a turn-based terminal interface so that sequencing is visible without introducing real-time scheduling or graphics APIs prematurely.

The update function should be callable without a terminal. A test can supply `d` and inspect the returned position. A replay can feed a sequence of commands. The interactive adapter can call the same function after reading input. This is a small engine-like boundary built from the pure functions you already know.

## Define a coordinate convention

Our map has ten columns and ten rows. Coordinates range from 0 through 9. The origin is the upper-left cell. x increases rightward and y increases downward. Thus w requests `(0,-1)`, a requests `(-1,0)`, s requests `(0,1)` and d requests `(1,0)`. These are chosen screen-grid conventions, not a universal definition of every mathematical or 3D coordinate system.

Write the convention beside the map. A movement error may be a sign mismatch rather than a broken loop. Test one step in each direction from an interior position so that an edge does not hide the intended change.

## Normalize at the input boundary

A player may type `D` or add spaces. The normalization function strips surrounding whitespace and converts the command to lowercase. It does not execute a game action. The rule then looks up the normalized command in a small movement table.

Unknown commands produce no movement in this helper model. The interactive adapter should explain the supported controls. Do not interpret arbitrary text as code or silently substitute a different command. The capstone also limits command length so accidental oversized input is rejected explicitly.

## Generate a candidate, then accept it

Calculate the candidate position before assigning it as the new state. If either coordinate is outside the map, reject the entire movement. Do not update x first and discover afterward that y is invalid. The whole-move rule makes failure predictable and prevents partial changes from leaking through rejection paths.

For the pure helper, returning the original tuple is a straightforward no-op. In a larger state object, other fields such as energy must also remain unchanged when the rule says a rejected move costs nothing. We will add that consistency requirement during integration rather than bury it inside drawing.

## Render by observation

A renderer walks the rows and columns and chooses a symbol for each cell. P marks the player, E marks the exit and a dot marks open space. When the player stands on the exit, give P display priority so there is still one visible player marker. The underlying game state, not the displayed symbol, determines victory.

Rendering returns a string. The terminal adapter prints it. A test can compare the string with a known small map and verify that repeated rendering does not change position. A 3x3 example is easier to audit than a 10x10 screenshot, while the actual lab still builds the specified 10x10 board.

## Assemble the smallest complete loop first

Begin with movement and display, then add input normalization and useful unknown-command feedback. Do not add physics, networking or animated graphics to avoid finishing a simple loop. Keep a written list of what the current stage does not yet implement: obstacles, energy and terminal win/loss policy arrive next. A clear limitation is better than a half-implemented feature claiming to be complete.

## Worked example

Predict before running `examples/example_05_1.py`.

```
position = (1, 1)
command = " D ".strip().lower()
deltas = {"w": (0, -1), "a": (-1, 0), "s": (0, 1), "d": (1, 0)}
dx, dy = deltas[command]
candidate = (position[0] + dx, position[1] + dy)
if 0 <= candidate[0] < 3 and 0 <= candidate[1] < 3:
    position = candidate
for y in range(3):
    row = ""
    for x in range(3):
        row += "P" if (x, y) == position else (
            "E" if (x, y) == (2, 2) else ".")
    print(row)
print("position after drawing:", position)
```

The example uses a 3x3 board for a compact trace. It performs exactly one candidate update, then only reads the result while producing rows. Drawing the rows again would not change the tuple. The exit is a display marker here; reaching it becomes a state transition in the next lesson.

## Expected demonstration output

```
...
..P
..E
position after drawing: (2, 1)
```

## Practical work and pass evidence

Implement `normalise_command`, `step_position` and `render_grid` in `labs/student/lab_05_1.py`. The default board is 10x10; the helpers also accept a small square size for tests. Draw P at the player and E at the bottom-right exit unless the player occupies it. Unknown or out-of-bounds movement returns the unchanged valid position.

Connect these helpers in a terminal adapter in your workspace. Test one movement per input and repeated drawing with no updates. Compare a known 3x3 rendering, then run the actual 10x10 version. Pass by showing that the update can be called without input or printing and that the display is read-only.

## Changed-case challenge

Add a second read-only display function that reports numeric coordinates. Calling both displays must still produce only one update per command.

## Companion source study

[S06] Python: More Control Flow Tools; [S07] Python: Data Structures; [S10] Python: Modules. Read the sections relevant to the named concepts; record the section studied and one additional example.

## Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. Are a simulation tick and a rendered frame necessarily identical?
2. What should happen when render is called twice without an update?
3. Why is headless simulation useful?
4. With an upper-left origin and y increasing downward, which coordinate change represents w/up?
5. Why should victory depend on the state rather than whether an E symbol remains visible after drawing?

**Evidence to retain:** predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

# Lesson 05.2: Collision rules and complete behavior

**Learning outcome:** Specify edge cases before implementing a playable rule.

## Turn a partial demonstration into a specified game

Movement on an empty board is not yet a complete game. Add an explicit goal, obstacles and rules for what happens after victory. The lesson model uses five fixed obstacles: (1,1), (3,2), (4,4), (7,6) and (8,8). The player starts at (0,0) and the goal is (9,9). A clear route exists along the top edge and then down the right edge.

Do not rely on the player to avoid unsupported situations. Define behavior at each edge, against a wall, for unknown commands and after the round ends. “Nothing should happen” must specify whether position, energy, turn count, messages or replay history are allowed to change. The helper model contains only x, y and won; the capstone expands that state explicitly.

## Reject the whole candidate

Construct a candidate coordinate pair and check it against both map bounds and obstacles before changing the authoritative world. A blocked movement leaves x, y and won unchanged. Return a new flat state dictionary so the original input remains available for comparison; do not mutate it and then try to undo selected fields.

This is a small transaction-like design pattern: prepare, validate, commit. We are not claiming database transaction guarantees; we are using the idea to make one in-memory operation easier to reason about. The pattern becomes more important as a movement also affects resources and status.

## Terminal states need a policy

When an accepted move reaches (9,9), set won=true. In the lesson helper, all later commands leave that world unchanged. The capstone adds explicit restart and quit behavior around its richer state machine. Keep the difference documented instead of accidentally making the helper's terminal policy inconsistent with its tests.

A restart should create a fresh initial state. Reusing a mutated starting dictionary can carry old information into the new round. A terminal status should also not cause a program to spin endlessly printing the same screen without input or exit handling. Separate the world transition from the terminal adapter's decision to leave the application.

## Add a loss condition for the capstone

The completed reference game begins with 25 energy. A legal move or wait spends one unit; rejected wall/edge moves and unknown commands spend none. Reaching the goal wins even when that last legal action spends the final unit. Otherwise energy zero causes a loss. After won/lost/quit, movement is a no-op; r creates a fresh round.

The order of goal and exhaustion checks matters. A state at the goal with zero energy is a win under the specified priority. Checking exhaustion first would produce a different game. Tests must include this tie case instead of assuming one outcome is universally correct.

## Invariants connect fields

For this capstone, remaining energy equals 25 minus accepted-action turn count. Both values must be nonnegative and the turn count cannot exceed 25 within one round. The player must stay on an open map cell. A won state must be at the goal; a lost state has zero energy and is not at the goal. These are relational requirements, not only type checks.

Do not add energy to the simpler lesson helper until it passes its own contract. Then integrate the richer rules in your capstone project with new tests. Incremental construction gives you a known-good baseline and isolates whether a failure came from collision handling, resource accounting or terminal-state policy.

## Worked example

Predict before running `examples/example_05_2.py`.

```
world = {"x": 1, "y": 0, "won": False}
obstacles = {(1, 1), (3, 2), (4, 4), (7, 6), (8, 8)}
candidate = (world["x"], world["y"] + 1)
next_world = dict(world)
if (0 <= candidate[0] < 10 and 0 <= candidate[1] < 10
        and candidate not in obstacles):
    next_world["x"], next_world["y"] = candidate
    next_world["won"] = candidate == (9, 9)
print("attempted cell:", candidate)
print("original:", world)
print("result:", next_world)
```

The attempted downward step hits the obstacle at (1,1). The candidate is rejected before any coordinate assignment, so the complete state remains unchanged. The new dictionary is independent at the flat-field level; the helper's fields are integers and a Boolean. This example does not yet contain the capstone energy fields.

## Expected demonstration output

```
attempted cell: (1, 1)
original: {'x': 1, 'y': 0, 'won': False}
result: {'x': 1, 'y': 0, 'won': False}
```

## Practical work and pass evidence

Implement `step_world` in `labs/student/lab_05_2.py` with the fixed map, pure flat-state return and terminal win policy. Test all four edges, all five obstacles, a normal move, reaching the goal and commands after victory. Preserve the input dictionary and reject the complete move rather than changing one coordinate early.

Once the helper passes, implement the capstone energy/loss/restart requirements from `CAPSTONE_SPEC.md` in your own game. Keep separate evidence for the helper and the richer game. Pass by explaining collision rejection, terminal priority and a live restart that leaves no stale round state.

## Changed-case challenge

Reach the goal on exactly the last energy unit by waiting seven times, moving right nine times and down nine times. Predict the final status before testing.

## Companion source study

[S06] Python: More Control Flow Tools; [S09] Python: Classes. Read the sections relevant to the named concepts; record the section studied and one additional example.

## Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What is missing from "move right increases x"?
2. Under the reject policy, what happens at x = 9 on a 0-9 grid?
3. Why validate before applying a multi-part state change?
4. Why is changing x before checking whether the full candidate cell is blocked risky?
5. Under the capstone rule, what status results when the final energy unit is spent entering the goal?

**Evidence to retain:** predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

# Lesson 05.3: Deterministic replay and simple measurement

**Learning outcome:** Reproduce a run and measure the work actually performed.

## Determinism is a claim about specified inputs

For our pure lesson model, the same initial state, map, rules and ordered commands should produce the same final state. This statement includes more than a list of keyboard letters. Changing the map, starting position or rule version changes the input conditions even when the command text is identical.

The lesson helper starts from a fixed world, so its replay function can accept only the command sequence. The capstone replay file is more explicit: it records a format version, rule/map identifier, initial state and ordered commands. This makes the replay's assumptions inspectable instead of depending on undocumented defaults.

## Replay tests more than the final screenshot

Record a twenty-command session and replay it several times. Compare state values, not just the appearance of the terminal. A different message history can produce the same final location; a renderer could also draw two different internal states identically. For this assignment, specify which state fields must agree and preserve the commands as data.

A canonical serialization puts fields in a stable order and uses a defined encoding. A digest of that serialization is a convenient compact comparison value. Equal digests are not a substitute for defining the state schema or proving correctness, and a process-dependent hash should not be treated as a portable saved identifier. The reference uses JSON with sorted keys and a SHA-256 digest for the lesson's small flat state.

## Remove hidden inputs where practical

Wall-clock time, nondeterministic event order and unrecorded random choices can make a replay diverge. Our lesson intentionally avoids them. If randomness is added later, record enough information to reproduce its effects and pin the relevant environment or random-generation policy; a bare seed is not a universal promise across every implementation and future rule change.

For the terminal capstone, save/load commands are interface operations rather than movement commands. A successful load starts a new replay segment from the loaded initial state. The bounded replay buffer also starts a new segment when full. These choices prevent a saved command list from pretending it began at a state that no longer matches the actual session.

## Measure a defined region

A benchmark should say what is timed. The supplied microbenchmark times 10,000 calls through the headless Python update loop by default, across seven trials after an untimed warm-up. It excludes terminal drawing and file I/O. It includes Python loop overhead, state validation, rule processing and message selection. That result is not whole-game frames per second.

The workload cycles through right, left, wait and restart so it does not spend almost every iteration doing a cheap terminal-state no-op. A benchmark that reaches victory on iteration 18 and then repeatedly returns unchanged state would measure mostly a different code path from active play. Representative workload design matters before interpreting the number.

## Preserve raw samples and avoid invented conclusions

Use a monotonic performance timer such as `time.perf_counter` for elapsed intervals. Keep every trial duration, the actual interpreter and platform, the command cycle and final state. Report a median as a summary while retaining the samples. A single fast run can reflect noise, and a mean alone can hide variability.

Do not claim one machine is faster than another without comparable workloads and measurement methods. Do not invent a required microsecond threshold for this educational game. The learning outcome is a defensible experiment: define the work, verify the result, measure repeatedly and limit the conclusion to what was actually tested.

## Worked example

Predict before running `examples/example_05_3.py`.

```
import hashlib
import json

first = {"x": 9, "y": 9, "won": True}
second = {"won": True, "y": 9, "x": 9}
first_text = json.dumps(first, sort_keys=True, separators=(",", ":"))
second_text = json.dumps(second, sort_keys=True, separators=(",", ":"))
first_digest = hashlib.sha256(first_text.encode("utf-8")).hexdigest()
second_digest = hashlib.sha256(second_text.encode("utf-8")).hexdigest()
print("same canonical text:", first_text == second_text)
print("same digest:", first_digest == second_digest)
print("canonical text:", first_text)
```

The dictionaries were written with different key orders but represent the same flat state. Sorting keys yields the same serialized text for this schema. The digest compares those defined bytes; it does not demonstrate that the movement leading to the state was correct. Keep independent expected transitions in the test suite.

## Expected demonstration output

```
same canonical text: True
same digest: True
canonical text: {"won":true,"x":9,"y":9}
```

## Practical work and pass evidence

Complete `labs/student/lab_05_3.py` after the Lesson 05.2 helper works. Replay an empty session, a twenty-command session and the known route to the goal. Compare both state fields and canonical digests across repeated runs. Preserve exact commands and rule assumptions.

Run `tools/benchmark_game.py` as a reference experiment, then adapt the same method to your own implementation without comparing unlike workloads. Keep the raw JSON report and state which operations are included. Pass by reproducing a session and explaining why the microbenchmark is not a whole-game FPS measurement.

## Changed-case challenge

Benchmark a deliberately poor workload that spends almost all iterations in a terminal state. Compare its timed operation with the active-play cycle before comparing durations.

## Companion source study

[S17] Python time library: perf_counter; [S11] Python json library; [S18] Python hashlib library. Read the sections relevant to the named concepts; record the section studied and one additional example.

## Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What information does a replay need besides button names?
2. Why exclude printing from a simulation microbenchmark?
3. Does one matching replay prove determinism for all inputs?
4. Why can a replay with identical commands diverge after a map or rule change?
5. Why is a loop that reaches victory early and then performs thousands of no-op updates a misleading active-play benchmark?

**Evidence to retain:** predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

# Lesson 05.4: Finishing, usability and the first portfolio artifact

**Learning outcome:** Deliver a tiny game another person can run and understand.

## Finish a small system rather than expand an unfinished one

A completed first game has a clear starting state, understandable controls, valid and invalid input behavior, a reachable success condition, a failure condition where specified, restart and a reliable way to leave. It can be built or run from documented source, tested and explained. It does not need commercial art, networking, an account system or a large feature list.

Use the Gate A specification as the acceptance boundary. Freeze optional feature ideas in a separate list while repairing missing required behavior. Adding enemies or an editor does not compensate for a save that corrupts state or a restart that carries over old energy. Scope control is part of engineering, not a sign of insufficient ambition.

## Test from a clean extraction

Create a release candidate in a fresh folder without your incidental workspace files. Follow the README exactly. Verify the interpreter check, launch command, controls, tests, save/load and replay behavior. A program that works only because your editor injected a path or an old file already exists is not reproducible from the package alone.

Do not bundle virtual environments, private credentials, machine-specific caches or unrelated documents. Include source, tests, required small data files, instructions and a clear list of known limitations. Keep generated saves in a separate workspace. Name the version and preserve a short change history connecting the release to the tested source.

## Ask a tester to use the instructions without coaching

Give another person the release candidate and README. Ask them to start, move, encounter a blocked action, finish or lose, restart, quit and run the tests. Observe where they hesitate or fail, without quietly supplying the missing step. Record three concrete observations and fix at least one usability issue.

A useful observation describes behavior: “the tester typed up because the controls were not visible” is better than “the tester was confused.” A possible repair is showing controls at launch and making help available. Verify the change with another clean attempt. Do not describe one informal session as a comprehensive accessibility or usability study.

## Accessibility begins with clear feedback

Use readable text, consistent controls and messages that distinguish rejection from success. Do not communicate a required event only by color or by a fleeting animation. The terminal reference uses explicit symbols and status text rather than depending on sound or color. That is a bounded design choice, not a guarantee that every terminal or assistive technology has been tested.

Record untested access needs honestly. A later game-production volume treats accessibility in greater depth. For this checkpoint, demonstrate that core instructions and outcomes are available as text and that the user can restart or quit without guessing hidden commands.

## Submit evidence, not only the game

The Gate A handoff includes your source, a rule specification, README, at least twelve learner-written tests, a saved replay with independently expected final state, a two-bug journal, a usability record and a short technical explanation. The supplied reference code may be studied, but submitting it unchanged does not demonstrate your implementation skill.

A reviewer should ask for one live rule change: perhaps a different starting-energy limit or one additional obstacle. Predict affected tests first. Explain what remains invariant and how save/replay compatibility is handled. A change that silently reinterprets an old replay under new rules is not an acceptable shortcut.

## State what completion means

Passing this volume means you can reason about, implement, test and hand off a small program with explicit state and behavior. It is not a claim that you are already a professional engine architect or that a degree has been awarded. Preserve the artifact as the baseline for future mathematics, native programming and systems work.

The next teaching volume in the program order is Mathematics for Games and Computing, Modules 06-10. Keep your current program small and working; later lessons can reuse it as a controlled place to investigate vectors, motion and measurement rather than replacing it with a much larger engine before you understand the foundations.

## Worked example

Predict before running `examples/example_05_4.py`.

```
from pathlib import Path
import subprocess
import sys

root = Path(__file__).resolve().parents[1]
command = [sys.executable, str(root / "capstone" / "reference" / "play.py"),
           "--demo"]
result = subprocess.run(command, capture_output=True, text=True,
                        timeout=10, check=True)
print(result.stdout.splitlines()[-1])
print("Reference demonstration completed; your own Gate A remains required.")
```

This example runs the completed reference game through a known winning sequence. The final status line is an example of release behavior a reviewer can reproduce; running the same command directly also displays the board. It is not a student submission, and the label states that distinction. The subprocess uses the same Python interpreter as the wrapper rather than assuming an unrelated launcher name.

## Expected demonstration output

```
Energy=7 Turns=18 Status=won
Reference demonstration completed; your own Gate A remains required.
```

## Practical work and pass evidence

Complete `CAPSTONE_SPEC.md` and the five-module project requirements. Build your own game under `capstone/student/` or an explicitly documented workspace. Replace the intentionally failing starter test with at least twelve independent tests and add a meaningful design change rather than merely submitting the reference.

Ask a tester to follow your README without coaching, record three observations and repair at least one issue. Submit the release evidence and demonstrate a live rule variation. The supplied game checker tests the reference, not your project; use your own test command and independent review for Gate A.

## Changed-case challenge

Prepare a handoff note explaining one known limitation, one tested failure path and one proposed next feature, each clearly separated from claims of completed work.

## Companion source study

[S01] Harvard CS50 Introduction to Programming with Python; [S12] Python unittest library; [S13] Pro Git: Recording Changes to the Repository. Read the sections relevant to the named concepts; record the section studied and one additional example.

## Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What belongs in a clean-start README?
2. Why should color not be the only important cue?
3. What evidence should accompany a claimed improvement?
4. The supplied reference game passes its tests. Does copying it unchanged satisfy the student Gate A implementation requirement?
5. A tester needs verbal instructions that are absent from the README. What should the release review record?

**Evidence to retain:** predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.